

# Query Processing

## Selections Involving Comparisons

---

### 1. Introduction

---

Selections involving comparison operators ( $\leq, \geq, <, >$ ) can be implemented using:

- A **linear file scan** (Algorithm A1)
- **Index-based scans** using clustering or secondary indices (A5, A6)

These are used to efficiently retrieve tuples satisfying:

$$\sigma_{A \leq v}(r), \quad \sigma_{A \geq v}(r), \quad \sigma_{A < v}(r), \quad \sigma_{A > v}(r)$$

#### Key Terminology

- $b_r$ : Total number of blocks in relation  $r$
- $b$ : Number of blocks containing matching records
- $h_i$ : Height of the B<sup>+</sup>-tree index
- $t_S$ : Time for one disk seek (random access latency)
- $t_T$ : Time to transfer one block
- $n$ : Number of matching records

### 2. A5: Clustering Index (Comparison)

---

**Applicable when:** The relation is physically sorted on attribute  $A$  and has a clustering (primary) index.

A clustering ordered index (e.g., B<sup>+</sup>-tree) enables efficient range searches for comparison conditions.

#### 2.1 For $\sigma_{A \geq v}(r)$ :

1. Use the index to locate the **first tuple** where  $A \geq v$ .
2. Sequentially scan the file from that point onward.

## 2.2 For $\sigma_{A \leq v}(r)$ :

1. Begin a sequential file scan from the start.
2. Stop once the **first tuple with**  $A > v$  is found.
3. The index is less effective here (used mainly to verify stopping point).

### Cost Formula

$$\text{Cost} = h_i \times (t_S + t_T) + t_S + b \times t_T$$

### Formula Breakdown:

1.  $h_i \times (t_S + t_T)$ : Traverse  $h_i$  index levels to find the starting point.
2.  $1 \times t_S$ : One seek to position at the first matching data block.
3.  $b \times t_T$ : Sequential transfer of the  $b$  blocks containing matching records.

### Example

**Relation:** Employee(EID, Name, Salary) **Index:** Clustering index on Salary.

**Query:**  $\sigma_{\text{Salary} \geq 50000}(\text{Employee})$

- Locate the first record where  $\text{Salary} \geq 50000$  using the B<sup>+</sup>-tree.
- Sequentially scan the remaining tuples in sorted order.

## 3. A6: Secondary Index (Comparison)

A secondary ordered index can be used for comparison operations ( $<$ ,  $\leq$ ,  $>$ ,  $\geq$ ) even if the relation is **not** sorted on  $A$ .

### 3.1 Procedure:

- For  $A \leq v$  or  $A < v$ : Scan index leaf nodes from the beginning up to  $v$ .
- For  $A \geq v$  or  $A > v$ : Start from  $v$  and scan to the end of the leaf nodes.
- Retrieve tuples using pointers from index entries.

### Cost Formula

$$\text{Cost} = (h_i + n) \times (t_S + t_T)$$

### Formula Breakdown:

1.  $h_i \times (t_S + t_T)$ : Traverse the index to find the starting leaf entry.
2.  $n \times (t_S + t_T)$ : Each of the  $n$  matching records requires a **separate random I/O** because data is not physically ordered by this key.

### Important Note

When  $n$  is large (many matching records), A6 can become **slower than a linear scan** (A1). This is because:

- Linear scan:  $b_r \times t_T + 1 \times t_S$  (mostly sequential)
- A6:  $n \times (t_S + t_T)$  (all random I/O)

If  $n \times t_S > b_r \times t_T$ , prefer linear scan.

### Example

**Relation:** Product(PID, Price) **Index:** Secondary index on Price.

**Query:**  $\sigma_{Price < 200}(Product)$

- Scan index leaf nodes from the start until reaching value 200.
- Fetch each record with  $Price < 200$  using index pointers.

## 4. A7: Hybrid Bitmap-Based Algorithm

When both full scans and index scans are costly, a **bitmap-based hybrid** approach offers balanced performance.

### 4.1 Algorithm Steps:

1. Scan the secondary index and mark blocks containing matches in a **bitmap**.
2. Sort the block addresses to enable sequential disk access.
3. Sequentially scan only those marked blocks.

### Advantages:

- **Best-case:** Much faster than pure secondary index scan (sequential I/O).
- **Worst-case:** Only slightly worse than linear scan (extra index lookup overhead).

### Example

**Query:**  $\sigma_{Age \leq 30}(Person)$

1. Use index on **Age** to identify block IDs containing matching records.
2. Create a bitmap marking those blocks.
3. Scan only the marked blocks sequentially.

## 5. Summary and Comparison

| Method          | I/O Pattern | Best For                       | Watch Out                   |
|-----------------|-------------|--------------------------------|-----------------------------|
| A5 (Clustering) | Mixed       | Range queries on sorted data   | Less effective for $\leq v$ |
| A6 (Secondary)  | Random      | Selective queries (small $n$ ) | Slow when $n$ is large      |
| A7 (Bitmap)     | Sequential  | Moderate selectivity           | Extra index scan overhead   |

### Key Takeaways

- **Clustering indices** are ideal for range-based selections on sorted data.
- **Secondary indices** work best for highly selective queries (small  $n$ ).
- **Linear scan** may outperform A6 when many records match.
- **Bitmap hybrid** offers balanced performance across different selectivities.